



Orca Test Configuration Guide

WALTER CAPERS



Orca Test Configuration Guide

Orca's configuration file gives you complete control over how tests are generated, executed, retried, triggered, and evaluated.

Each configuration is deterministic (seeded random), repayable, and scriptable.

This guide explains:

- All fields in the config schema
- When to use each field
- How RANDOM and SEQUENCE events work together
- How failures, retries, and triggers behave
- Example configurations (basic → advanced)

Overview

Orca executes tests based on JSON-defined configuration files. An Orca configuration file describes **which events should run, how they should run, and the conditions under which they are allowed to run**. It contains two layers of control:

Global controls, which shape the behavior of the entire test session (randomness, debugging behavior, failure policy, logging, etc.)

Event-level controls, which apply only to the specific event where they are defined and override global settings when necessary.

An **Orca event** is any executable action—such as a script, ADB command, reboot operation, or system interaction—that can run on the host or device under test. Each event can specify its own execution mode (sequential, random, manual), preconditions, post-actions, retry logic, and behavior on failure.

Together, these elements form a flexible and deterministic framework for building sophisticated stress, reliability, and workflow tests.

Top-Level Configuration Fields

An Orca configuration file begins with a set of **global controls** that affect all events unless overridden at the event level. These fields shape the entire behavior of the stress engine.

```
{
  "randomSeed": 12345,           // deterministic randomness
  "targetPackage": "com.app",    // package monitored for process health
  "defaultRetry": { ... },      // fallback retry policy
  "defaultSlowThresholdMillis": 8000,
  "debug": false,               // enables debug_logger.debug()
  "debugBreakBeforeEvent": false, // step mode
  "debugBreakOnRetry": false,
  "debugBreakOnError": false,
  "events": [ ... ],           // full event list
  "logcat": { ... }            // log capture config
}
```

randomSeed (number)

Purpose: Ensures deterministic randomness across runs.

How it works: Orca's event selection uses a pseudo-random generator. By fixing the seed, Orca guarantees the exact same sequence of events (including retries, delays, and triggers) when replaying a run.

Use when:

- You want reproducible tests
- You want to replay failures deterministically
- You are debugging event ordering

Example:

```
"randomSeed": 987654321
```

targetPackage (string)

Purpose: Identifies the main application under test.

How Orca uses it:

- Checks process health after each event
- Restarts the app after reboot events if configured
- Starts the app at the beginning of duration-based runs

Use when: Always. This is a required field.

Example:

```
"targetPackage": "com.example.myapp"
```

defaultRetry (object)

Defines the **fallback retry strategy** for events that do not specify their own.

Fields:

```
"defaultRetry": {  
  "maxAttempts": 2,  
  "backoffSeconds": 3,  
  "strategy": "LINEAR"    // or EXPONENTIAL  
}
```

Used when an event does not have an `event.retryPolicy`.

Use when:

- You want consistent retry behavior across all events
- You don't want to repeat retry settings in every event

Events may override this with:

```
"retryPolicy": { ... }
```

defaultSlowThresholdMillis (number)

Purpose: Emits a warning when an event takes longer than this value.

This helps highlight performance regressions or hangs.

Example:

```
"defaultSlowThresholdMillis": 2500
```

Use when:

- You want visibility into slow operations
- You want to detect degradation under stress

Events may override with their own `slowThresholdMillis`.

***debug* (boolean)**

Purpose: Enables verbose debug logging (`logger.debug(...)`).

This does *not* affect breakpoints—only log verbosity.

Use when:

- You are debugging state transitions
- You want to see detailed engine decisions
- You want visibility into pre/post metrics

Example:

```
"debug": true
```

Interactive Debug Break Flags

These fields control step-mode and breakpoint behavior during execution. These flags enable the interactive debugging of Orca configuration files-scripts.

***debugBreakBeforeEvent* (boolean)**

Purpose: Enables global **step-mode**.

When true:

- The engine pauses **before every event**
- A prompt appears: *step / continue / quit*
- The user may inspect state

Use when:

- Building new configs
- Debugging event ordering
- Understanding replay behavior

debugBreakOnRetry (**boolean**)

Pauses execution anytime an event is about to retry.

Useful for:

- Diagnosing flaky events
- Inspecting the cause of retry loops
- Watching incremental backoff behavior

debugBreakOnError (**boolean**)

Pauses execution **immediately after a failed event attempt**.

Useful when:

- You want to inspect failure details interactively
- You need to see partial metrics or stdout/stderr

stopOnFailure (**boolean**)

Purpose: Stops the entire test run on the first failure, regardless of event-level `onFailure`.

Difference from `onFailure`: **STOP_TEST**:

Behavior	<code>stopOnFailure</code> (global)	<code>onFailure: STOP_TEST</code> (event-level)
Scope	Applies to <i>all events</i>	Applies only to <i>this event</i>
Activation	Triggers on any failure	Triggers only if <i>this event</i> fails
Intended use	Quick abort for safety	Granular per-event test control
Config location	Top-level	Inside event definition

Use `stopOnFailure` when you want a strict test run that halts immediately upon a failure of any event.

events (**array**)

This is where the full event list is defined.

Each entry may be:

- `RANDOM event`
- `SEQUENCE event`

- `MANUAL event`
- `NO_OP`
- `WAIT_FOR_DEVICE`
- `SCRIPT event`

Each event can define its own policies, triggers, metrics, and conditional logic.

Example:

```
"events": [  
  {  
    "id": "warmup",  
    "type": "SCRIPT",  
    "mode": "SEQUENTIAL",  
    "script": { "inline": ["echo warming up"] }  
  }  
]
```

logcat (**object**)

Configures logcat capture behavior.

Example:

```
"logcat": {  
  "enabled": true,  
  "rotateOnReboot": true,  
  "tag": "MyApp"  
}
```

Fields:

Field	Purpose
enabled	If false, disables logcat entirely
rotateOnReboot	Stops logcat before reboot + restarts afterward
tag	Tag or package used to filter logcat

Use when:

- You want diagnostics from Android logs
- You want per-event timing correlations
- You want to debug state after crashes or reboots

Summary Table (Quick Reference)

Field	Level	Description
randomSeed	Global	Makes randomness reproducible

<code>targetPackage</code>	Global	App under test
<code>defaultRetry</code>	Global	Fallback retry logic
<code>defaultSlowThresholdMillis</code>	Global	Warn on slow events
<code>debug</code>	Global	Debug logging
<code>debugBreakBeforeEvent</code>	Global	Step mode
<code>debugBreakOnRetry</code>	Global	Break when retrying
<code>debugBreakOnError</code>	Global	Break on failure
<code>stopOnFailure</code>	Global	Abort entire run if <i>any</i> event fails
<code>events</code>	Global	Event definitions
<code>logcat</code>	Global	Logcat capture settings

Event Objects

Orca supports **three event execution modes**—**RANDOM**, **SEQUENCE**, and **MANUAL**—and all three modes may be freely mixed within the same configuration. This flexibility allows test designers to create simple linear workflows, completely randomized stress scenarios, or hybrid models where deterministic sequences coexist with randomized event selection.

Every entry in the `"events"` array defines an **Orca event**, which is the fundamental unit of test behavior.

An event may execute a script, trigger a sequence, impose conditions, or manage device reboot flow.

Orca supports multiple event types:

- **SCRIPT**
- **SEQUENCE**
- **RANDOM**
- **MANUAL**
- **NO_OP**
- **WAIT_FOR_DEVICE**


```
{
  "id": "network_stress",
  "description": "Performs a network-intensive stress event",
  "type": "SCRIPT",
  "mode": "RANDOM",
  "enabled": true,
  "weight": 4,
  "profileWeights": { "low_end": 10, "high_end": 2 },
  "script": {
    "inline": [
      "curl -s https://example.com > /dev/null"
    ]
  },
  "preconditions": {
    "networkRequired": true,
    "batteryAbove": 25
  },
  "retryPolicy": {
    "maxAttempts": 3,
    "backoffSeconds": 2,
    "strategy": "EXPONENTIAL"
  },
  "onFailure": "RETRY",
  "slowThresholdMillis": 1500,
  "metrics": {
    "captureCpuUsage": true,
    "customMetrics": ["netLatency"]
  },
},
"postEvents": ["evaluate_network_health"],
"conditionalTriggers": [
  {
    "triggerEventId": "latency_recovery",
    "ifMetricAbove": { "netLatency": 300 }
  }
],

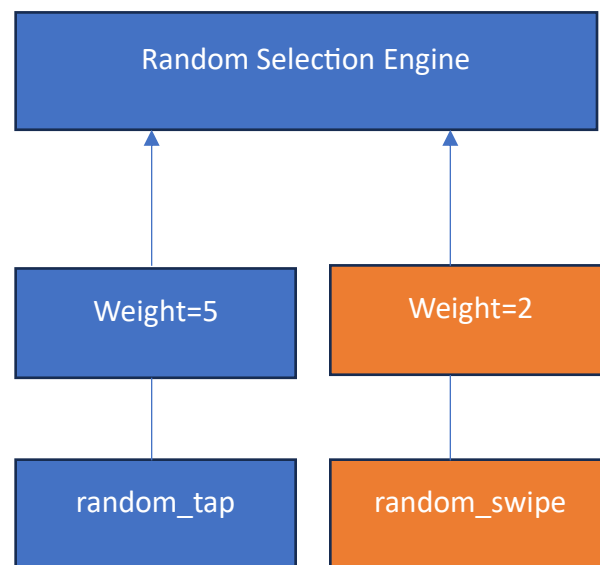
"debugBreak": false,
"tags": ["network", "latency"]
}
```

Orca Event Types

RANDOM Events

Participate in the weighted random selection loop.

- Chosen according to weights
- Retries applied automatically
- Can trigger other events
- Post-events run after success



SEQUENCE Events

Not random. Executed only when explicitly triggered:

warmup_sequence

└─ screen_on

└─ launch_app

└─ health_check

MANUAL Events

Executed only when referenced by:

- A sequence
- A post-event
- A conditional trigger
- A SEQUENCE or SEQUENCE child
- A direct call from the CLI or API

Event Object- Required Fields

id (string)

Unique identifier for the event.

Used in:

- sequences
- conditional triggers
- postEvents
- summary reporting
- replay failure traces

Example:

```
"id": "warmup_start"
```

type (string)

Defines what the event *does*. Allowed values:

Type	Description
SCRIPT	Runs a script (inline or file). Most common event type.
SEQUENCE	Runs multiple events back-to-back.
NO_OP	Does nothing; useful for state setting or placeholder behavior.
WAIT_FOR_DEVICE	Handles reboot → offline → online → boot complete flow.
MANUAL (optional feature)	Runs only when invoked interactively.

Example:

```
"type": "SCRIPT"
```

mode (**string**)

Defines **how the engine selects the event**:

Mode	Behavior
RANDOM	Included in weighted random selection (default).
SEQUENTIAL	Executed only when referenced in a SEQUENCE or manual flow.
CONDITIONAL	Triggered by conditionalTriggers only.
MANUAL	Only runs in interactive mode; not selected automatically.

Example:

```
"mode": "RANDOM"
```

Basic Event Control

enabled (**boolean**, default: true)

Disables an event without removing it.

Useful for:

- temporarily turning off unstable events
- A/B configuration testing
- OS-version–dependent events

description (**string**)

Human-readable explanation. Used in:

- summaries
- documentation
- interactive debug messages

tags (**array<string>**)

Arbitrary labels to categorize events.

Example uses:

- "startup"

- "network"
- "battery"
- "critical"

Tags may be used in future filtering/selection logic.

Random Selection Behavior

These fields control *weighted probability* during RANDOM selection.

weight (number)

Probability weight in RANDOM mode.

Example, If weights are:

Event Weight

A 1

B 4

B is selected 80% of the time.

If omitted, defaults to **1**.

profileWeights (object<string, number>)

Overrides weight per performance/profile category.

Example:

```
"profileWeights": {  
  "low_end": 10,  
  "high_end": 2  
}
```

This allows the same config to behave differently for device tiers.

Script Execution Controls

These fields apply only when "type": "SCRIPT".

script (**object**)

Defines what the script does. Two modes:

Inline:

```
"script": {  
  "inline": ["echo hello", "sleep 1"]  
}
```

File-based:

```
"script": {  
  "file": "scripts/reboot.sh"  
}
```

Supported script languages:

Language	Field value
Shell / Bash	"SHELL"
Python	"PYTHON"
Windows Batch	"BATCH"

language (**string**)

Explicit interpreter selection.

Retry Logic

retryPolicy (**object**)

Overrides global defaultRetry.

Fields:

```
"retryPolicy": {  
  "maxAttempts": 3,  
  "backoffSeconds": 2,  
  "strategy": "EXPONENTIAL"  
}
```

onFailure (enum)

Event-level failure behavior:

Value	Meaning
RETRY	Retry until retryPolicy is exhausted
STOP_TEST	Abort the <i>entire test</i> upon this event's failure
SKIP_FUTURE	Disable this event for the remainder of the run
LOG_ONLY	Record failure, but continue normally

Slow Event Threshold

slowThresholdMillis (number)

Warns when an event exceeds this duration.

Overrides global `defaultSlowThresholdMillis`.

Preconditions and State Controls

preconditions (object)

Allows dynamic enable/disable of events based on system state:

Supported keys:

Condition	Meaning
networkRequired	Event only runs if network is available
batteryAbove	Only run if battery > value
batteryBelow	Battery < value
rootRequired	Device must be rooted
adbAvailable	ADB must be available
fileMustExist	Array of required file paths

Example:

```
"preconditions": {  
  "networkRequired": true,  
  "batteryAbove": 20
```

```
}
```

requireState (**object**)

Event only runs when engine state matches this map.

Example:

```
"requireState": { "phase": "warmup_done" }
```

setState (**object**)

Sets one or more state variables when event succeeds.

Example:

```
"setState": { "phase": "stress_phase" }
```

Chaining, Triggers, and Conditional Logic

postEvents (**array**)

Runs additional events *after* success.

Example:

```
"postEvents": ["verify_health", "cleanup_temp"]
```

Executes each event normally through the policy engine.

conditionalTriggers (**array<object>**)

Runs an event when a condition matches.

Supported conditions:

Condition	Meaning
ifOutputContains	Match stdout/stderr substring
ifExitCodeNotZero	Trigger if script fails
ifExitCodeEquals	Trigger if script returns specific code
ifMetricAbove	Trigger if metric exceeds threshold
ifMetricBelow	Trigger if metric is below threshold

Example:

```
"conditionalTriggers": [  
  {  
    "triggerEventId": "latency_recovery",  
    "ifMetricAbove": { "netLatency": 300 }  
  }  
]
```

Reboot Event Controls

These apply only when the event triggers a device reboot.

causesReboot (**boolean**)

If true:

- Device offline → online → boot completed cycle is handled
- Logcat rotates if configured
- App restarts if configured
-

waitForBoot (**boolean**)

Waits for BOOT_COMPLETED broadcast.

restartAppAfterBoot (**boolean**)

Re-launches the target app immediately after reboot.

Manual and Debug-Mode Fields

debugBreak (**boolean**)

Event-level breakpoint.

Pauses *before* executing this event even if global breakpoints are disabled.

Metrics Collection

metrics (object)

Defines what metrics to collect before/after execution.

Supported:

- `captureCpuUsage`
- `captureMemoryUsage`
- `captureBatteryLevel`
- `customMetrics`

Example:

```
"metrics": {  
  "captureCpuUsage": true,  
  "customMetrics": ["netLatency"]  
}
```

Results merge into:

- global metrics summary
- event-level `ScriptResult` metrics

Event-Level vs Global Controls: Quick Comparison

Behavior	Global	Event-Level
Retry behavior	<code>defaultRetry</code>	<code>retryPolicy</code>
Slow threshold	<code>defaultSlowThresholdMillis</code>	<code>slowThresholdMillis</code>
Breakpoints	<code>debugBreakBeforeEvent</code>	<code>debugBreak</code>
Failure handling	<code>stopOnFailure</code>	<code>onFailure</code>
Metrics	<code>none</code>	<code>metrics</code>
Preconditions	<code>none</code>	<code>preconditions, requireState</code>
State writing	<code>none</code>	<code>setState</code>

How to Design Good Orca Events

Designing effective and maintainable Orca events is the core to building reliable stress-testing scenarios.

This guide provides **best practices** for structuring events, using modes intelligently, balancing probabilities, designing sequences, applying preconditions, and handling failures gracefully.

The goal is not only to help you write *correct* configs, but to help you write configs that are *predictable, debuggable, deterministic*, and meaningful.

Start With a Clear Testing Strategy

Before writing a single event, decide:

What fault domains are you stressing?

- Network?
- Storage and file operations?
- CPU / memory load?
- Application state transitions?
- Reboot recovery?
- Environmental transitions (screen, charging, battery)?
-

What are the primary questions?

Examples:

- “Does the app survive intermittent network failures?”
- “Does the system recover after repeated reboots?”
- “Do we leak memory over repeated large operations?”
- “Do multiple app launches produce inconsistent results?”

What outcomes do you care about?

- App stability?
- System recovery?
- Performance degradation?
- Unexpected exit codes?
- Conditional trigger behavior?

Use Event Types Intentionally

Orca supports multiple event types, all with different roles:

Event Type	Best Use Case
SCRIPT	Primary action: bash, Python, ADB, batch scripts
SEQUENCE	Enforcing strict execution order
RANDOM mode	Weighted non-deterministic exploration
NO_OP	State changes, rhythm control, marking transitions
WAIT_FOR_DEVICE	Reboot sequences, recovery checkpoints

Use `SEQUENCE` for structured flow

Examples:

- Warmup sequences
- Startup initialization
- Tear-down tasks
- Multi-step failure simulations

Use `RANDOM` for chaos-based testing

Examples:

- Flaky network toggling
- Randomized CPU pressure
- Intermittent low-memory triggers
- Repeated rapid app relaunches

Use `MANUAL` for debugging

Interactive, human-driven steps.

Keep Events Focused (One Responsibility Each)

Rule of thumb:

A well-designed event should do *one thing only* and do it clearly.

Examples:

- `toggle_wifi`
- `simulate_latency`
- `verify_network_health`
- `restart_app`

- `clean_cache`
- `pull_logs`
- `warmup_1, warmup_2`

Events become easier to track in summaries, easier to replay, and easier to refine.

Avoid “mega events” that:

- perform multiple unrelated operations
- mix stress, verification, and cleanup
- perform nested workflows manually

Use `postEvents` or `SEQUENCE` instead.

Use Preconditions to Avoid Invalid Behavior

Preconditions prevent meaningless failures and stabilize test behavior.

Examples:

Battery constraints

```
"preconditions": { "batteryAbove": 20 }
```

Require screen on/off state

```
"preconditions": { "screenOn": true }
```

Require network for network tests

```
"preconditions": { "networkRequired": true }
```

Require root for privileged ops

```
"preconditions": { "rootRequired": true }
```

Require ADB availability

```
"preconditions": { "adbAvailable": true }
```

When preconditions fail

Orca **skips the event**, rather than marking it a failure.

This prevents *false negatives* in your final summary.

Use State Machines for Complex Logic

Orca supports a very powerful internal state machine via:

- `requireState`
- `setState`

Use it to enforce order without sequences.

Example: warmup → stress → cleanup

```
{
  "id": "warmup_completed",
  "setState": { "phase": "warmup_done" }
}
{
  "id": "start_stress",
  "requireState": { "phase": "warmup_done" }
}
{
  "id": "cleanup",
  "requireState": { "phase": "stress_complete" }
}
```

Benefits:

- Clear phases
- Deterministic replay
- Skip irrelevant events
- Avoid failures in inappropriate states
-

Apply Failure Policies Correctly

Use `onFailure` wisely:

Policy	Best Use Case
LOG_ONLY	Non-critical events, exploratory chaos tests
RETRY	Network flakiness, intermittent issues
SKIP_FUTURE	Permanently broken event under current conditions
STOP_TEST	Critical failures that invalidate future behavior

Global vs Event-Level

Global:

```
"stopOnFailure": true
```

— the run ends on **any** event failure.

Event level:

```
"onFailure": "STOP_TEST"
```

— the run ends *only* when this event fails.

Avoid mixing STOP_TEST on events that run randomly unless you *intend* possible early termination.

Keep Weights Balanced

Weights shape the *probability distribution* of RANDOM events.

Good pattern:

- Many exploratory events with low weight
- Few core events with higher weight
- Rare, destructive events with very small weight

Example:

```
"weight": 1           // default exploratory
"weight": 3           // main flows
"weight": 10          // warmups during startup
"weight": 0 or disabled // extremely destructive
```

Avoid:

- Having a single event with a weight 100x larger than others
- Extremely rare events that never run but matter

Use Conditional Triggers for Smart Self-Recovery

Conditional triggers allow events to fire *based on results*:

Examples:

Retry health checks

```
"ifExitCodeNotZero": true
```

React to metrics

```
"ifMetricAbove": { "cpu.totalPercent": 90.0 }
```

Respond to log messages

```
"ifOutputContains": "CRITICAL_ERROR"
```

Best use cases:

- Adaptive stress testing
- Auto-recovery routines
- Stateful multi-step failures
- Intelligent exploration
-

Handle Reboots Properly

Whenever you use events that cause actual reboots:

```
"causesReboot": true
```

Make sure you:

- Provide a `WAIT_FOR_DEVICE` event afterwards
- Control logcat rotation
- Use `restartAppAfterBoot` if needed

Reboots without `WAIT_FOR_DEVICE` will result in unpredictable behavior.

Always Provide Helpful Descriptions and Tags

Descriptions show up in:

- step mode
- summaries
- logs
- debugging output

Good example:

```
"description": "Simulates high network latency using tc qdisc"
```

Tags help with:

- filtering
- debugging
- future tooling

Example:

```
"tags": ["network", "latency", "chaos"]
```

When in Doubt, Use Step Mode

Enable:

```
"debugBreakBeforeEvent": true
```

Or event-level:

```
"debugBreak": true
```

Step mode helps you:

validate scripts

- watch sequence behavior
- inspect state transitions
- confirm preconditions
- see metrics
- understand failure policies

This is often the *fastest way* to debug a new test plan.

Keep Event Names and IDs Meaningful

Avoid IDs like:

- "step1"
- "testscript_bash_1"
- "exp_event_022"

Prefer:

- "warmup_sequence"
- "toggle_network"
- "verify_memory_health"

- "chaos_reboot"
- "stress_cpu_high_load"

Event names become your documentation—keep them clean.

Avoid Hidden Side Effects in Scripts

A script should not:

- secretly reboot the device
- kill processes unless declared
- mutate state not announced
- produce inconsistent output
- change global device properties without being advertised

Use `causesReboot`, `setState`, `clear descriptions`, and `tags`.

Use Metrics Sparingly but Intentionally

Good metrics:

- CPU before/after expensive tasks
- Memory deltas after heavy allocations
- Battery deltas over long-running events

Avoid:

- collecting too many metrics every event
- collecting metrics for NO_OP events
- collecting metrics when they do not matter

Metrics are powerful—but only when used sparingly and with purpose.

Review Summary Outputs After Each Test

Your summary tells you:

- failures
- execution counts
- slow events
- disabled events
- global metric averages
- final state
- RNG usage and deterministic replay information

Use this to tune:

- weights
- preconditions
- retry strategies
- what should or should not be STOP_TEST
- slow thresholds